

PRACTICAL WEEK 12

Task 1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

1) Create a file

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ mkdir Practical_week12
saichaitanya@GeethzzSai:~$ cd Practical_week12
saichaitanya@GeethzzSai:~/Practical_week12$ nano 1.c
```

CODE:

```
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t lock;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);

        pthread_mutex_lock(&lock);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
```

```

        pthread_mutex_lock(&lock);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&lock);
        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&lock, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&lock);

    printf("\nProducer-Consumer completed successfully.\n");

    return 0;
}

```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/f  ×  +  v
saichaitanya@GeethzzSai:~$ mkdir Practical_week12
saichaitanya@GeethzzSai:~$ cd Practical_week12
saichaitanya@GeethzzSai:~/Practical_week12$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week12$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week12$ ./1
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Produced: 11
Consumed: 11
Produced: 12
Produced: 13
Consumed: 12
Consumed: 13
Produced: 14
Produced: 15
Produced: 16
Produced: 17
Produced: 18
Consumed: 14
Consumed: 15
Consumed: 16
Consumed: 17
Consumed: 18
```

```
Consumed: 10
Produced: 11
Consumed: 11
Produced: 12
Produced: 13
Consumed: 12
Consumed: 13
Produced: 14
Produced: 15
Produced: 16
Produced: 17
Produced: 18
Consumed: 14
Consumed: 15
Consumed: 16
Consumed: 17
Consumed: 18
Produced: 19
Produced: 20
Consumed: 19
Consumed: 20
```

Producer-Consumer completed successfully.

```
saichaitanya@GeethzzSai:~/Practical_week12$ |
```

Task-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

CODE:

```
saichaitanya@GeethzzSai: ~/f × + v
GNU nano 8.7.1 2.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    pthread_mutex_lock(&A);
    printf("Thread 1 locked A\n");

    sleep(1);

    pthread_mutex_lock(&B);
    printf("Thread 1 locked B\n");

    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&B);
    printf("Thread 2 locked B\n");

    sleep(1);

    pthread_mutex_lock(&A);
    printf("Thread 2 locked A\n");

    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);

    return NULL;
}
```

```
int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/Practical_week12$
saichaitanya@GeethzzSai:~/Practical_week12$ nano 2.c
saichaitanya@GeethzzSai:~/Practical_week12$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/Practical_week12$ ./2
Thread 2 locked B
Thread 1 locked A
```